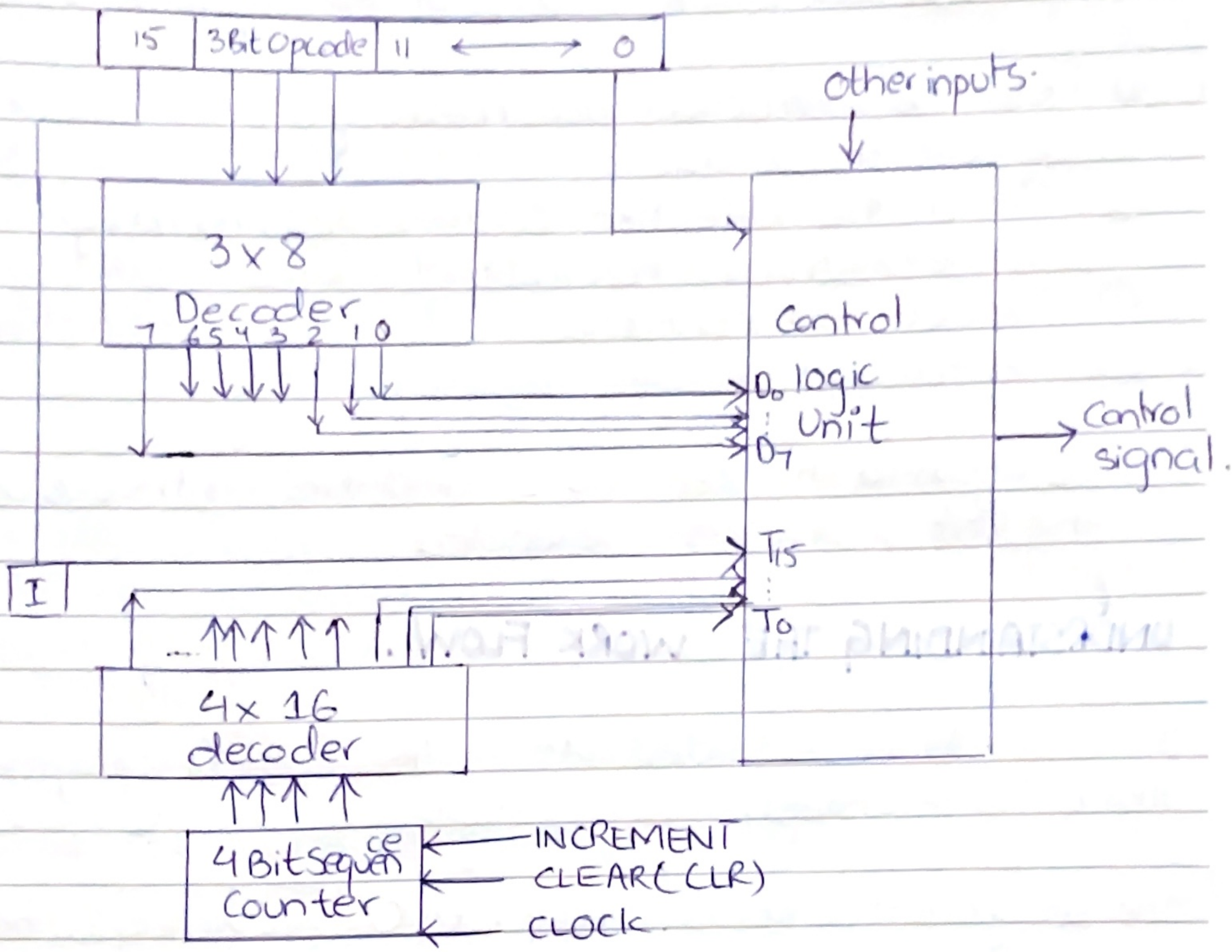
Date



INSTRUCTION CYCLE :-

A program residing in the memory unit of computer consists of a sequence of instruction. The program in the computer is executed by going through a cycle of I/for each instruction.

Each instruction cycle in turn is sub-divided into a sequence of subcycles or phases, from fetching of instruction to the completion of execution of instruction whatever happens is called the instruction cycle.

*The reason we called it a cycle is because it happens in every instruction.

Recall: Micro-op is a ... that occurs in/during one clock period. The result of this may replace binary info of register or result may be transferred into another register

Date

Each instruction cycle consists of the following phases.

① Fetch the instruction from memory.
② Decode the instruction.
③ Read the effective address from memory if the instruction has indirect address.
④ Execute the instruction.
⑤ Fetch and Decode. Again

i.e again to the step no. 1 and this continuous until the last instruction is executed.

UNDERSTANDING THE WORK FLOW.

Initially the PC is loaded with address of the first program that is to be runned.

The sequence counter is cleared to 0, providing a timing signal of 0.

After each clock pulse, SC is incremented by 1, so that the timing signal can go through a sequence of T_0 till T_{15}.

At $T_0 \rightarrow T_2$ the fetching and decoding occurs, here the control signals are basically accessing fetching the instruction and decoding it thus these 3 signals are common for all instructions. (i.e)

T_0 : $AR \leftarrow PC$
T_1 : $IR \leftarrow M[AR]$, $PC \leftarrow PC+1$

Date

T_3 : $D_0 \dots D_7 \leftarrow$ Decode (IR 12 – 14), AR ← (IR 4 0)
$I \leftarrow IR(15)$

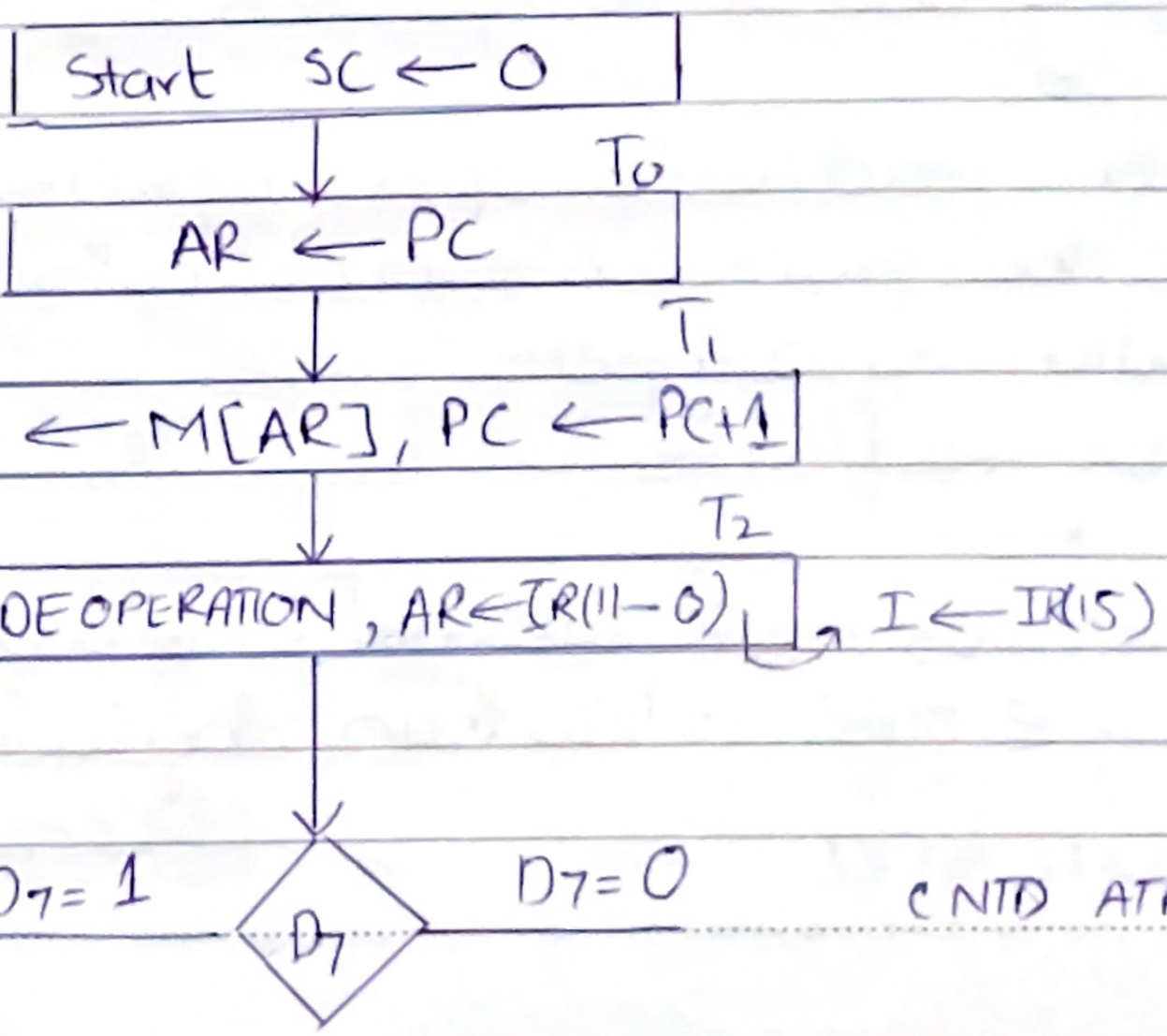
Now when we specified the addressing mode and got the operand from memory; we have entered the execution phase where it is important to execute what we know from the Opcode. ↓ and decoding of Opcode to know what to do

* Remember actual execution of our Instruction at this execution phase will need certain micro operations; which will be performed in certain clock pulse (we can't know how much exactly as it is d/f for d/f instruction, having their own micro-instructions) thus T_3 'included' & onwards till T_n are specific for execution.

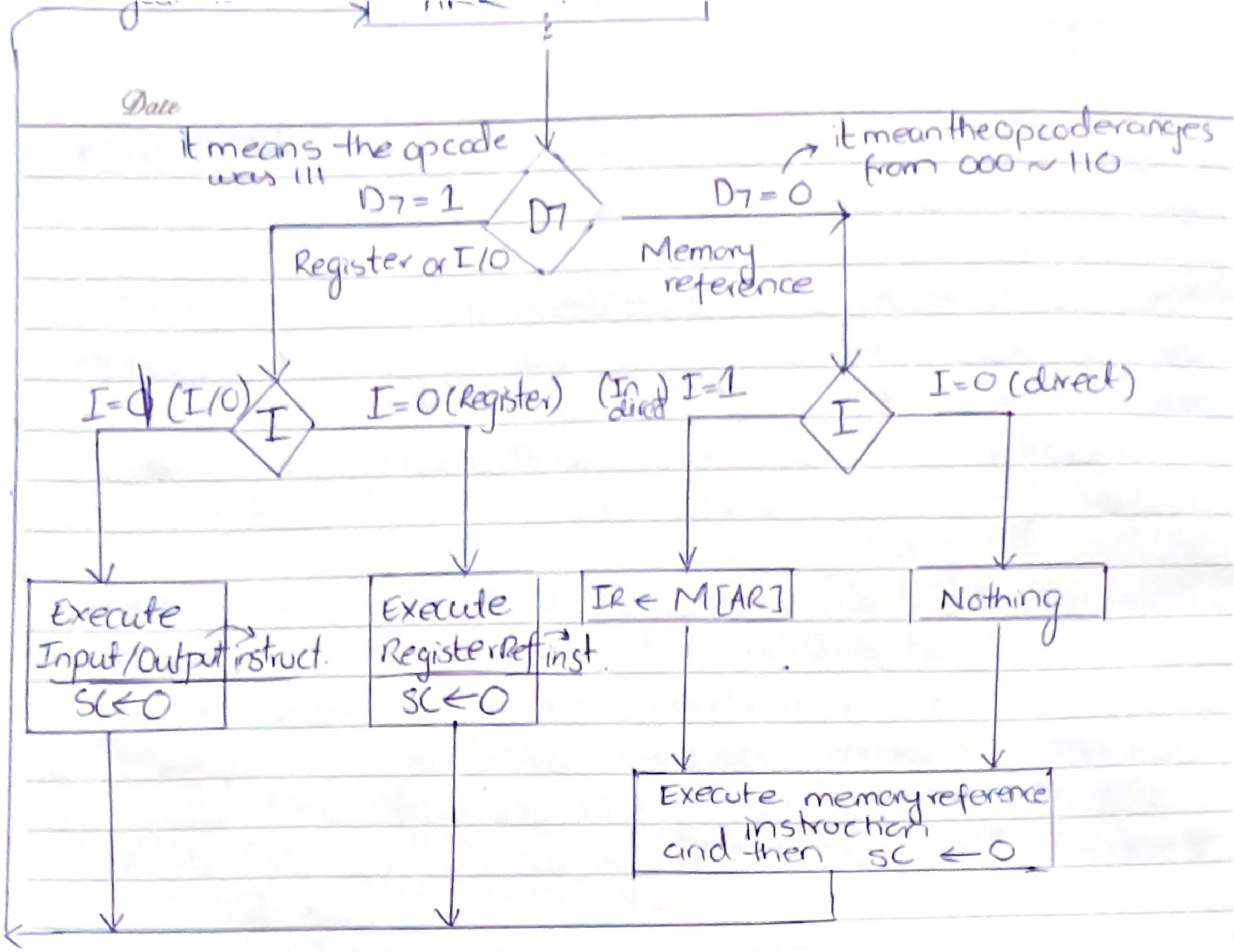
~x~

* Note at every T_n (after) a control signal generated performs certain task fetch, decode etc...

~x~



CNTD AT NEXT PAGE!



IMPORTANT TO KNOW

Normally in modern processors, the processor is trying to complete one basic operation of instruction in one clock cycle.

However read/write operations require several cycles. Therefore the processor can do nothing but wait for the the operation to complete
↓
memory read/write.

In moris mano's book we make the assumption that a memory cycle time is less than the clock time i.e.: no waiting for processor or CPU.
↓ (read/write time) ↓ (processor time)

Date

FETCH & DECODE CYCLE :-

We studied that fetch and decode cycle is common for every instruction and it ranges from $T_0 - T_2$

T_0 : $AR \leftarrow PC$ $(S_2S_1S_0 = 010\ T_0 = 1)$

T_1 = $IR \leftarrow M[AR]$ $PC \leftarrow PC+1$ $(S_2S_1S_0 = 111$

T_2 = $D_0 \ldots D_7 \leftarrow$ Decode IR(12-14), $(T_{01} = 1)$

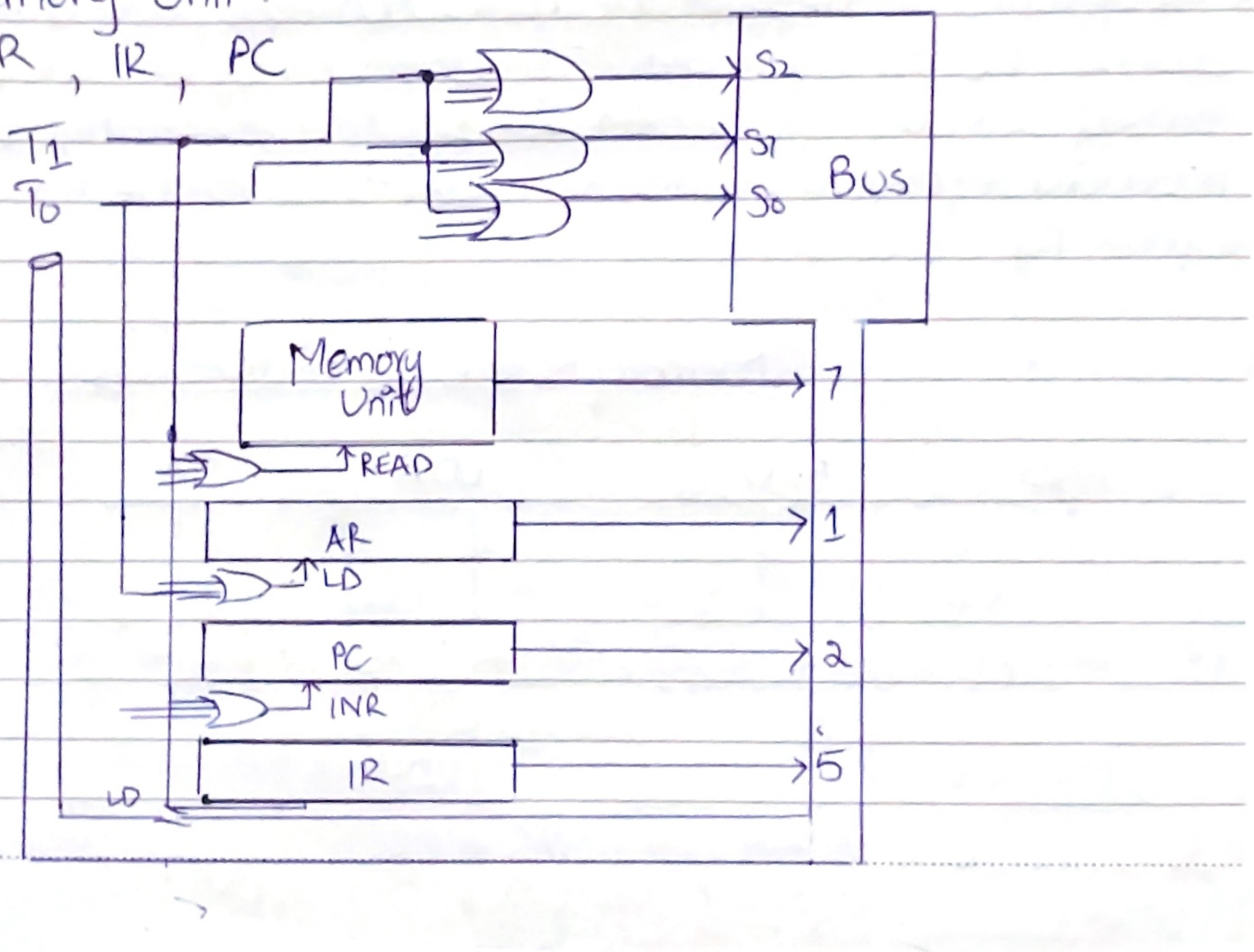
$AR \leftarrow IR(0-11)$, $I \leftarrow IR(15)$

for showing this cycles we demonstrate only T_0 & T_1 along with the following important component.

(i) A common BUS.

(ii) Memory unit.

(iii) AR, IR, PC



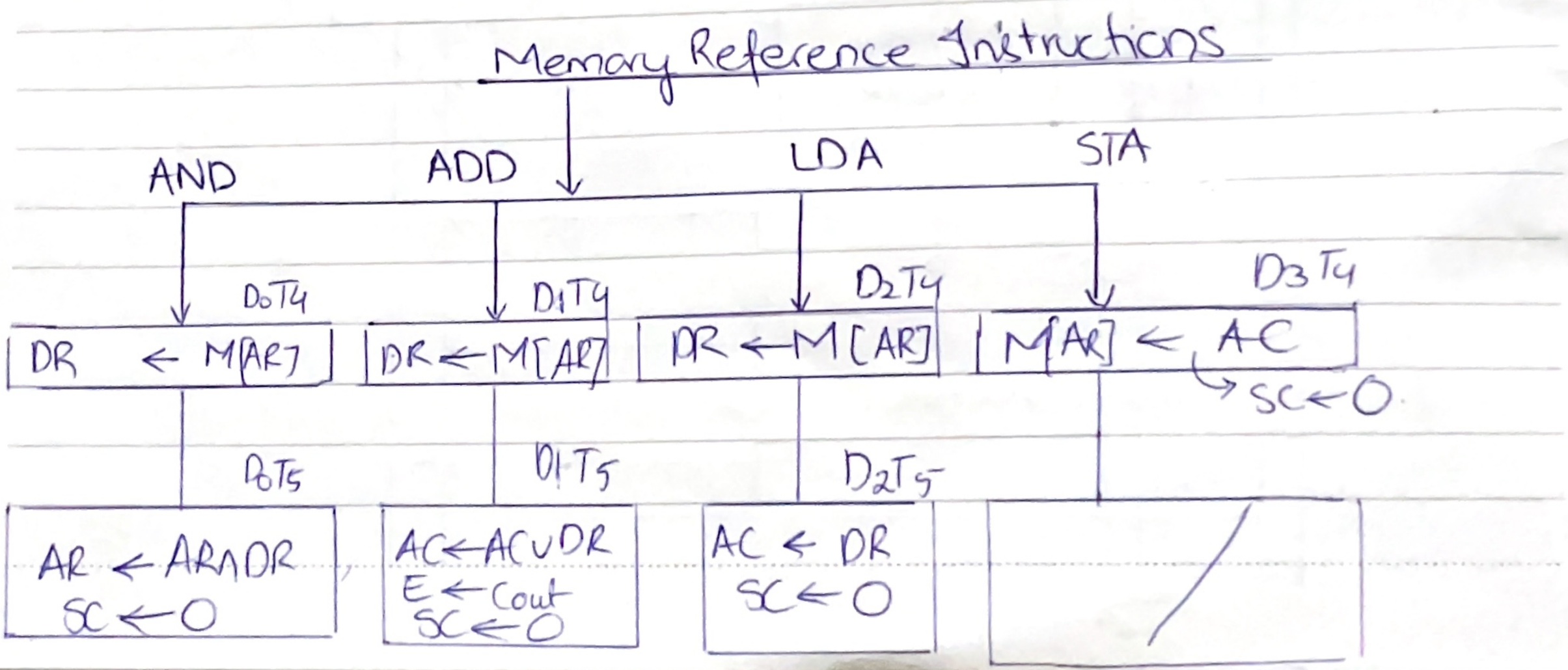
Date

MEMORY-REFERENCE INSTRUCTIONS.

SYMBOL	OPERATION DECODER	Symbolic Description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow Cout$
LDA	D_2	$AC \leftarrow M[AR]$
STA (Storeaccumulator)	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR+1$
ISZA	D_6	$M[AR] \leftarrow M[AR]+1,$ If $M[AR]+1 = 0$ then $PC = PC+1$.

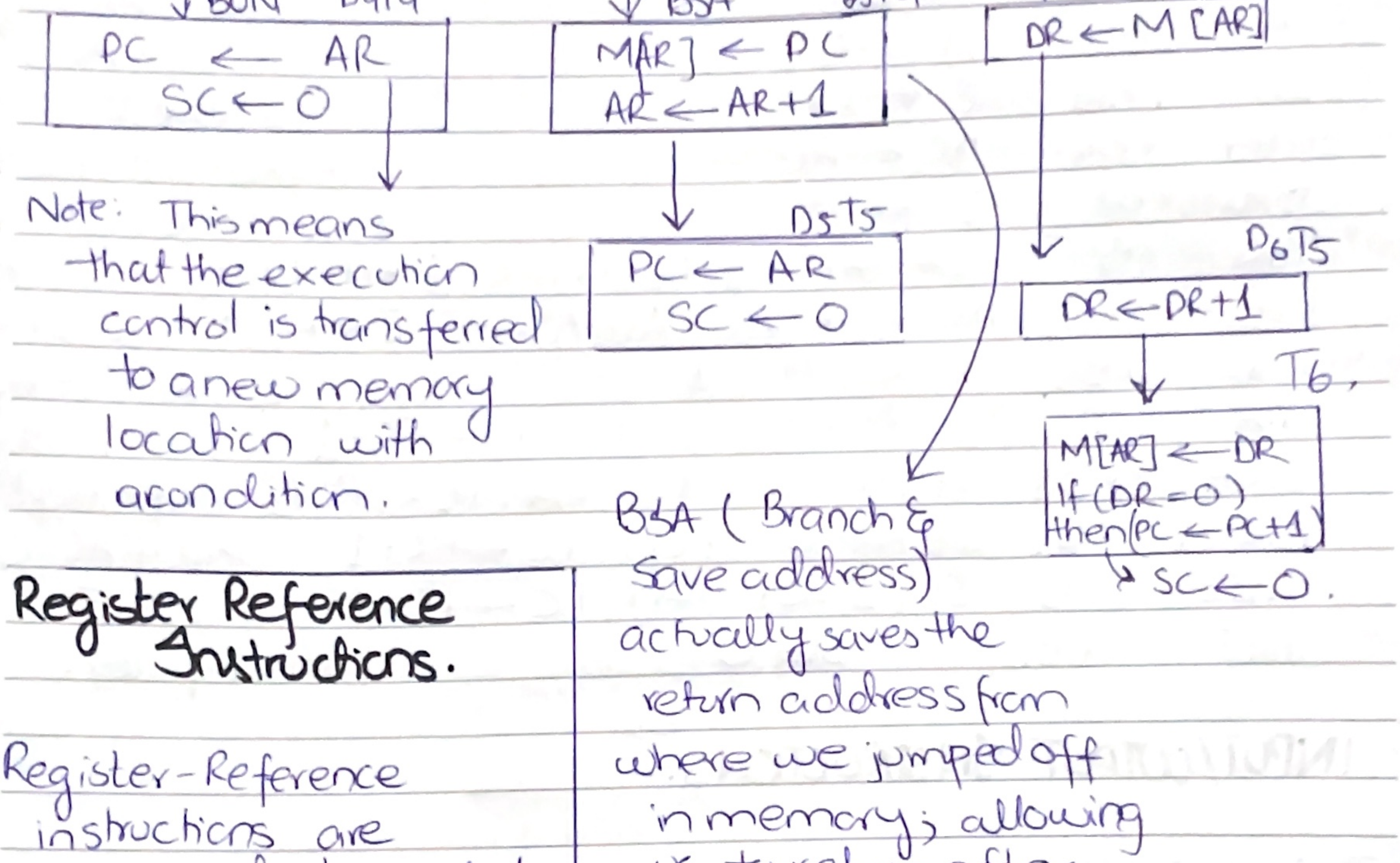
BUN → Branch unconditionally

* The effective address of the instruction was placed in AR during the timing signal T_2 when $I=0$, or during timing signal T_3 when $I=1$. The execution of the memory reference instruction starts with the timing signal T_4.



This instruction allows a program to branch to another memory location to execute an instruction that is not in the usual sequence.

Memory Reference Instructions.



Note: This means that the execution control is transferred to a new memory location with a condition.

BSA (Branch & Save address) actually saves the return address from where we jumped off in memory; allowing us to return after the execution of other location.

Register Reference Instructions.

Register-Reference instructions are recognized by control $D_7 = 1$. and $I = 0$.

These instructions use bit 0 through 11 of the instruction code to specify one of 12 instructions. (which are available through IR (11-0))

These instructions are executed with the clock transition associated with timing variable T_3.

Each control function needs Boolean relation $D_7 I' T_3$.

Date

$D_7 I' T_3 = r$ (common for all Register reference inst)
$IR(i) = B_i$ [bit in IR(0-11) that specifies the operation).

r: SC ← 0 clear SC

CLA	rB_{11}	AC ← 0	clear AC
CLE	rB_{10}	E ← 0	clear E
CMA	rB_9	AC ← $\overline{AC}$	complement AC
CME	rB_8	E ← $\overline{E}$	complement E
CIR	rB_7	AC ← shr AC, AC(15) ← E, E ← AC(0)	circulate right
CIL	rB_6	AC ← shl AC, AC(0) ← E, E ← AC(15)	circulate left
INC	rB_5	AC ← AC + 1	Increment AC
SPA	rB_4	If AC(15) = 0 then PC ← PC + 1	skip if positive
SNA	rB_3	If AC(15) = 1 then PC ← PC + 1	skip if negative
SZA	rB_2	If AC = 0 then PC ← PC + 1	skip if AC = 0
SZE	rB_1	If (E = 0) then PC ← PC + 1	skip if E zero
HLT	rB_0	S ← 0	halt computer.

INPUT/OUTPUT INSTRUCTIONS:-

The terminal sends and receive serial information. Each quantity of information has eight Bits of an alphanumeric code.

e.g:
The serial information from the keyboard is shifted into the input register INPR.

The serial information for the printer is stored in the Output register OUTR.

Date

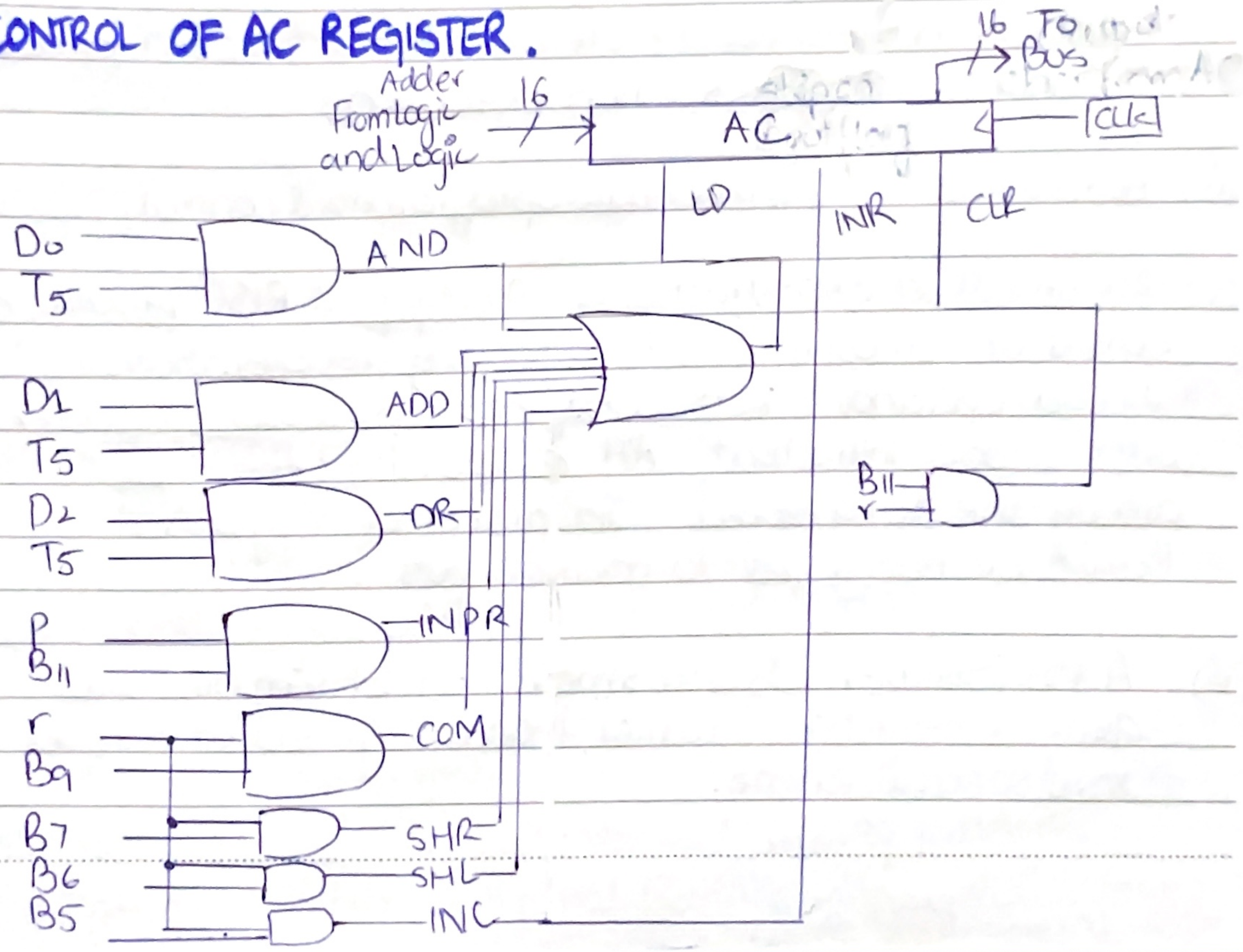
The 2 registers communicate interface serially and with the AC in parallel:

$D_7 IT_3 = P$

$IR(i) = B_i \rightarrow i = 6 \cdots 11$

	p:	SC ← 0	Clear SC
INP	p·B_{11}:	AC(0-7) ← INPR, FGI ← 0	Input char to AC
OUT	p B_{10}:	OUTR ← AC(0-7), FGO ← 0	Output char from AC
SKI	p B_9:	if (FGI = 1) then PC ← PC+1	Skip on input flag
SKO	p B_8:	if (FGO = 1) then PC ← PC+1	Skip on output flag
ION	p B_7:	IEN ← 1	Interrupt enable on
IOF	p B_6:	IEN ← 0	Interrupt enable off

CONTROL OF AC REGISTER.



Date → extremely fast (200 peta flops) → ARM architecture processors
seen in iphone/Androids/ipads etc.

REDUCED INSTRUCTION SET COMPUTER (RISC)

Computer that use fewer instructions with simple construct so they can be executed much faster within the CPU having to use memory more often are classified as RISC.

↓ without

Properties:-

(i) Relatively few instructions and few instruction modes.

(ii) Memory access is limited to load and store instructions as all operations done within the register's CPU.

(iii) fixed length and easily decodeable instruction format. (single cycle for instruction execution).

(iv) Hardwired rather than microprogrammed control.

(v) The small set of instructions of a typical RISC processor consist of mostly register-to-register operations. Thus each operand is brought into a processor register with a load instruction. All computations are done among the data stored in processor registers. Results are transferred to memory by means of store instruction.

(vi) A RISC computer has a small set of simple and general instruction rather than a large set of complex and specialized one.

Date

COMPLEX INSTRUCTION SET COMPUTER (CISC)

A computer with a large no. of instructions is classified as a complex instruction set computer.

(i) It has a large no. of instructions typically from 100-250 instructions.

(ii) Some instruction that perform specialized task are used frequently, unlike RISC which itself has only a few inst. which themselves are used frequently.

(iii) A large variety of addressing modes typically from 5-20.

(iv) Variable length of instruction formats.

(v) Consist of instructions that manipulate operands in memory.

NOTE IMP DETAIL:-

CISC is a computer in which single instruction can execute several low level microoperations (such as load, arthemetic and store) or they are capable of multi-step operations or addressing modes with in single instructions.

A modern RISC can be more complex than a CISC; in terms of the complexity of circuits, no. of instruction encoding patterns.

* :- differing characteristic is only that RISC use uniform instruction lengths for all inst. and employ strictly separate load/store inst.

Date

Q. Consider an hypothetical control unit implemented by hardware. It uses 3, 8-Bit Register A, B, C.

It supports two Instructions I_1, I_2; the following table gives control signals required for each micro-operation for both instructions.

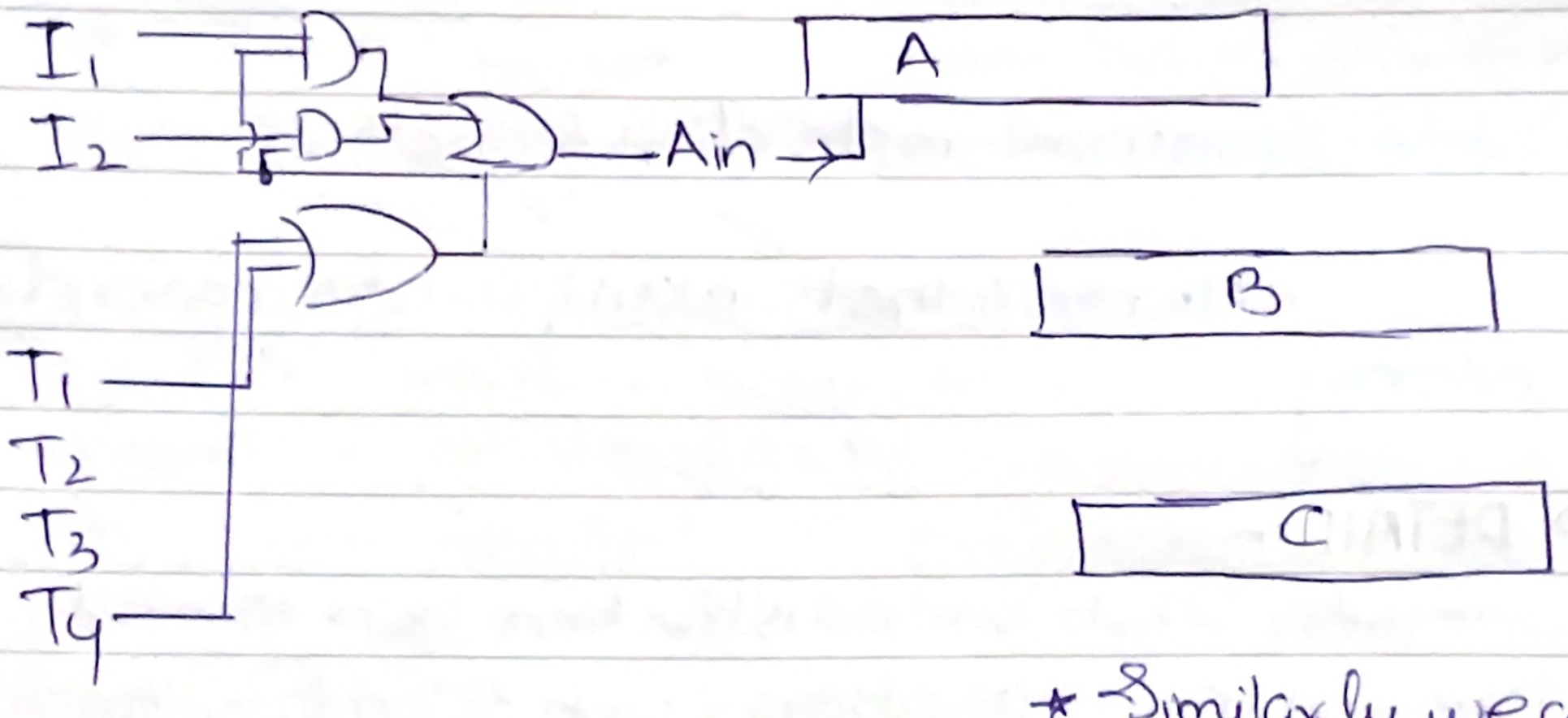
Given: ⟶

MICRO-OP	CONTROL SIGNAL	
	I_1	I_2
T_1	Ain Bout	Ain Aout
T_2	Bin Cout	Cin Aout
T_3	Bin Bout	Cin Cout
T_4	Ain Aout	Ain Bout

Solution.

Consider Ain

$$Ain = I_1T_1 + I_2T_2 + \bar{I}_1T_4 + I_2T_4.$$
$$= I_1(T_1 + T_4) + I_2(T_1 + T_4)$$



* Similarly we can make for all Ain(s)/outs) of B & C too.

Date

MICRO-PROGRAMMED CONTROL UNIT:-

The idea of microprogramming was introduced by Maurice Willkes, as an intermediate level to execute computer program instructions

Microprograms were organized as a sequence of micro-instructions and stored in special control memory

(i) Simple structure
(ii) Outputs of controller are organized in micro-instructions and they can be easily be replaced.
(iii) Binary patterns of control signals are stored in memory, hardware is used to generate the control signals (after access from memory)

Based on the no. of words in the control word, the microprogram may be

- Horizontal microprogram.
- Vertical microprogram.

Dif b/w Hardwired & Micro-programmed Control.

Characteristic	Hardwired	Micro-programmed control
Speed	Fast	slow
Implement	Hardware	software
flexibility	Not flexible	flexible
Ability to handle complex inst	Difficult	Easy.
Designing	Difficult for complex op.	easier.
Memory	Not used	Control Memory Used.

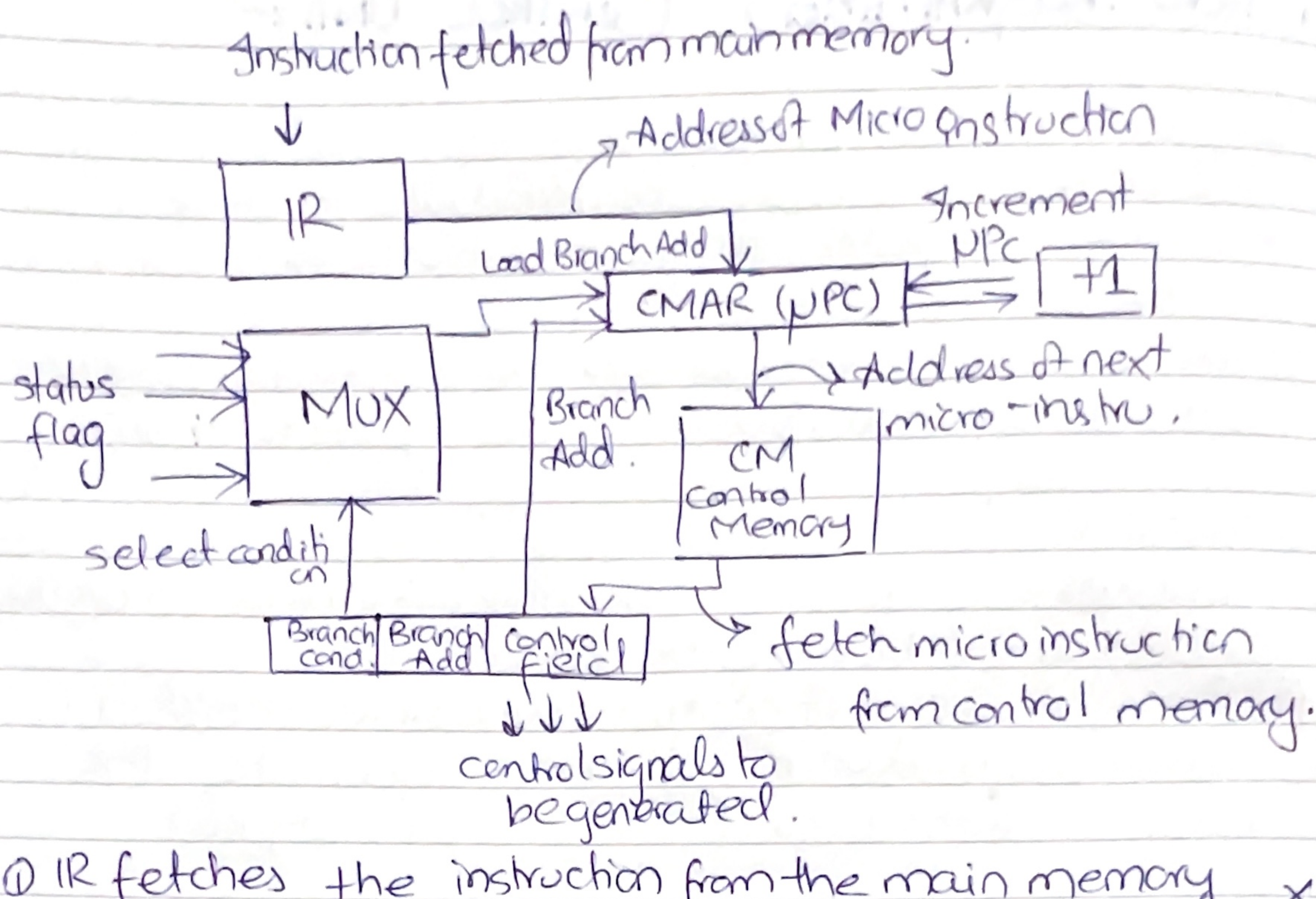
RISC uses hard wire

CISC uses microprogrammed

Uses longer micro instructions where each bit directly corresponds to a control signal, allowing multiple operations to be executed in parallel.

Date

HORIZONTAL MICRO-PROGRAMMED CONTROL UNIT.



① IR fetches the instruction from the main memory and provides the address of the first (micro-operation) micro-instruction to CMAR.

② CMAR fetches the corresponding instruction from control Memory CM.

③ The control memory provides control signals to execute the operations.

④ The μPC is either
- incremented normally (+1) to move to the next micro-instruction or updated with a branch address if a condition is met.

Date

(5) MUX selects the input based on the status flags to determine if a branch should occur.

(6) The cycle continues until the execution of instruction is complete.

VERTICAL MICROPROGRAMMED CONTROL UNT.

A vertical Microprogrammed Control Unit uses compact micro-instructions is encoded into fields, requiring further decoding to generate the actual control signals.

This often allows only one or few microinstructions per microinstruction, making execution more sequential.

~ X ~

	HORIZONTAL MICROPROGRAMMED CONTROL UNIT	VERTICAL MICROPROGRAMMED CONTROL UNIT.
①	long format	Short format
②	Ability to express high degree of parallelism.	limited ability " " "
③	little encoding of control info	Considerable encoding of control info.
④	useful when higher operating speed is desired	" slower operating speed is " .

* The combination of horizontal microprogram and vertical microprogram control unit is preferred in most applications.